



Unidad I: Software Libre y GNU/Linux

1.1 Software Libre

1.1.1 ¿Qué es el Software Propietario?

Antes de entrar a definir el software libre, veamos algunas características del software propietario mediante un ejemplo.

Imagina que vas a comprar un coche y las condiciones de compra son:

Ud. sólo puede circular con su coche por la provincia en la que reside. Si quisiera circular por otra provincia diferente necesitaría pagar más dinero en concepto de licencia.

No podrá ceder ni alquilar su coche.

No podrá modificarlo de ninguna manera, no podrá ponerle otro radio-cassette, colgarle unos dados del retrovisor, cambiarle los neumáticos, etc. Para hacerlo tendrá que solicitarlo al vendedor y obviamente le cobrarán por ello, y al sólo poder hacer las modificaciones el vendedor ¿se imagina cómo serán las tarifas?

No podrá abrirlo/desmontarlo para estudiar su funcionamiento.

¿Compraría un coche en estas condiciones? Seguro que no. Entonces, ¿cuál es la razón de comprar software propietario bajo unas condiciones similares?

Cuando compra un software propietario, si se molesta en leer la licencia que lo acompaña, verá que:

Sólo podrá instalar el software en un determinado número de equipos, requiriendo el pago adicional, en concepto de licencias, si quisiera instalarlo en más equipos.

Ud. no puede ceder ni alquilar el software que acaba de comprar.

No puede modificarlo de ninguna manera. El único que puede hacerlo es el desarrollador y en las condiciones que considere oportunas (y siempre y cuando le salga rentable).

No podrá realizar ingeniería inversa para estudiar su comportamiento.

1.1.2 ¿Qué es el Software Libre?

El "Software Libre" es un asunto de libertad, no de precio. Para entender el concepto, debes pensar en "libre" como en "libertad de expresión", no como en "barra libre" en inglés una misma palabra (free) significa tanto libre como gratis, lo que ha dado lugar a cierta confusión.

"Software Libre" se refiere a la libertad de los usuarios para ejecutar, copiar, distribuir, estudiar, cambiar y mejorar el software. De modo más preciso, se refiere a cuatro libertades de los usuarios del software:

La libertad de usar el programa, con cualquier propósito (libertad 0).

La libertad de estudiar cómo funciona el programa, y adaptarlo a tus necesidades (libertad 1). El acceso al código fuente es una condición previa para esto.

La libertad de distribuir copias, con lo que puedes ayudar a tu vecino (libertad 2).

La libertad de mejorar el programa y hacer públicas las mejoras a los demás, de modo que toda la comunidad se beneficie. (libertad 3). El acceso al código fuente es un requisito previo para esto.

Un programa es software libre si los usuarios tienen todas estas libertades. Así pues, deberías tener la libertad de distribuir copias, sea con o sin modificaciones, sea gratis o cobrando una cantidad por la distribución, a cualquier persona en cualquier lugar. El ser libre de hacer esto significa (entre otras cosas) que no tienes que pedir o pagar permisos.

También deberías tener la libertad de hacer modificaciones y utilizarlas de manera privada en tu trabajo u ocio, sin ni siquiera tener que anunciar que dichas modificaciones existen. Si



publicas tus cambios, no tienes por qué avisar a nadie, ni de ninguna manera en particular.

La libertad para usar un programa significa la libertad para cualquier persona u organización de usarlo en cualquier tipo de sistema informático, para cualquier clase de trabajo, y sin tener obligación de comunicárselo al desarrollador ni a ninguna otra entidad específica.

La libertad de distribuir copias debe incluir tanto las formas binarias o ejecutables del programa como su código fuente, sean versiones modificadas o sin modificar (distribuir programas de modo ejecutable es necesario para que los sistemas operativos libres sean fáciles de instalar). Está bien si no hay manera de producir un binario o ejecutable de un programa concreto (ya que algunos lenguajes no tienen esta capacidad), pero debes tener la libertad de distribuir estos formatos si encontraras o desarrollaras la manera de crearlos.

Para que las libertades de hacer modificaciones y de publicar versiones mejoradas tengan sentido, debes tener acceso al código fuente del programa. Por lo tanto, la posibilidad de acceder al código fuente es una condición necesaria para el software libre.

Para que estas libertades sean reales, deben ser irrevocables mientras no hagas nada incorrecto; si el desarrollador del software tiene el poder de revocar la licencia aunque no le hayas dado motivos, el software no es libre.

Así pues, quizás hayas pagado para obtener copias de software GNU, o tal vez las hayas obtenido sin ningún coste.

Pero independientemente de cómo hayas conseguido tus copias, siempre tienes la libertad de copiar y modificar el software, e incluso de vender copias.

"Software libre" no significa "no comercial". Un programa libre debe estar disponible para uso comercial, desarrollo comercial y distribución comercial. El desarrollo comercial del software libre ha dejado de ser inusual; el software comercial libre es muy importante.

A veces las normas de control de exportación del gobierno y las sanciones mercantiles pueden restringir tu libertad de distribuir copias de los programas a nivel internacional. Los desarrolladores de software no tienen el poder de eliminar o sobrepasar estas restricciones, pero lo que pueden y deben hacer es rehusar el imponerlas como condiciones de uso del programa. De esta manera, las restricciones no afectarán a actividades y gente fuera de las jurisdicciones de estos gobiernos.

Por último, fíjate en que los criterios establecidos en esta definición de software libre requieren pensarse cuidadosamente para interpretarlos. Para decidir si una licencia de software concreta es una licencia de software libre, lo juzgamos basándonos en estos criterios para determinar si tanto su espíritu como su letra en particular los cumplen. Si una licencia incluye restricciones contrarias a nuestra ética, la rechazamos, aun cuando no hubiéramos previsto el problema en estos criterios. A veces un requisito de una licencia plantea una situación que necesita de una reflexión minuciosa, e incluso conversaciones con un abogado, antes de que se pueda decidir si la exigencia es aceptable. Cuando llegamos a una conclusión, a veces actualizamos estos criterios para que sea más fácil ver por qué ciertas licencias se pueden calificar o no como de software libre.

Por lo general, para decidir si un determinado software es libre, puedes hacerte las siguientes preguntas:

- ¿Te dan las fuentes del programa?
- ¿Puedes modificar esas fuentes?
- ¿Puedes distribuir lo que modifiques?
- ¿Puedes vender esas modificaciones al precio que quieras?
- ¿Debes añadir las fuentes, obligatoriamente, al distribuir?

Según la FSF si la respuesta a las cuatro primeras es afirmativa el programa es software libre, si además la quinta es también positiva, entonces, será libre y con "copyleft".



1.1.3 Control y Seguridad

Cuando adquirimos un software propietario rara vez se nos suministra el código fuente. Sin embargo, la mayoría de la gente no le da importancia a este hecho ya que, como ellos dicen, "¿Y qué me importa a mí que me den el código fuente si no tengo los conocimientos necesarios para leerlo?".

Sin embargo, la única forma de poder fiarnos de la seguridad de un programa informático es tener a nuestra disposición el código fuente, ya que de esta manera podemos ver cómo ha sido programado y si lo ha sido de forma correcta. Además, también podremos comprobar que en dicho programa no hay:

Puertas traseras: No es necesario que el desarrollador las incluya. Hay veces en las que un intruso introduce una puerta trasera sin el conocimiento del desarrollador; otras veces la competencia puede pagar a un programador descontento para que introduzca esa puerta trasera.

Funcionalidades no documentadas: Un programa puede realizar ciertas tareas de las que no somos conscientes. Por ejemplo, un programa para cifrado de correo electrónico tendrá acceso a nuestras claves y podría haber sido programado para mandar esas claves a una determinada persona. O bien podría incluir información sobre nosotros y nuestras claves en las firmas digitales que utilicemos mediante la inclusión de canales subliminales.

La única forma de detectar todo esto es mediante la disponibilidad del código fuente, ya que encontrar un bug en un programa de ordenador no es tan sencillo como se piensa, a menos que se disponga del código fuente. La disponibilidad del código fuente nos da más seguridad en el sentido de transparencia: como el código fuente está disponible se puede auditar y comprobar así que está libre de puertas traseras y/o funcionalidades no documentadas, ya que en caso de tenerlas se descubrirían, y su hallazgo sería una auténtica vergüenza para la empresa que lo vende, pudiéndola llevar a la ruina.

Para conservar la libertad de la información, es imprescindible el uso de estándares abiertos. Esto es, estándares establecidos por entidades internacionales e independientes de las empresas particulares (ISO, IEEE, 3W, etc).

De otra manera dependeríamos de los intereses de la empresa que comercializa el software. Siendo este el único que puede leer nuestra información y no siendo posible que otras entidades escriban software alternativo, estaríamos vendidos.

1.1.4 ¿Cómo funciona?

El énfasis del sistema está en la colaboración. Se trata de una comunidad de usuarios/desarrolladores unida por un fin común. En el fondo todos son egoístas, trabajan en el proyecto porque lo usan y les interesa que funcione lo mejor posible y solucione sus propios problemas. De esta manera, todos se benefician.

1.1.5 Historia de un Proyecto

Alguna persona o entidad comienza un proyecto para satisfacer una necesidad propia. Por lo general no tienen que empezar de cero ni hacer todo el trabajo, hay ya otros proyectos que hacen cosas similares y bibliotecas de funciones que hacen parte del trabajo. No tienen más que coger lo que les interese de entre todo el software disponible a nivel mundial y mirar como se han resuelto otros problemas similares.

Cuando el proyecto se hace público, otra gente puede utilizarlo, encontrar deficiencias y corregirlas.

Cuando se han hecho modificaciones, se tratan de integrar con el proyecto original, y son los autores originales los que deciden si los cambios se aceptan o no.

Si se aceptan los cambios, el proyecto mejora y la historia se repite.

En el caso de que los cambios sean rechazados, el autor de estos puede o bien mantener los cambios independientemente del



proyecto original, o iniciar un nuevo proyecto (bifurcación/fork).

1.1.6. Modelos de desarrollo

Catedral

Es el modelo tradicional de desarrollo de software. De este tipo son las técnicas de desarrollo que se estudian en las asignaturas de Ingeniería del Software y se usan en las empresas.

Sus principales características son:

- Paso a paso, avances pequeños.
- Siguiendo un diseño de un arquitecto magistral
- Gran secreto
- Grandes recursos
- Solo se deja entrar a los feligreses una vez terminada

Bazar

Es el modelo mas habitual en software libre. A menudo es considerado inviable por los expertos en ingeniería del software, pero el hecho es que funciona.

Sus principales características son:

- Gran número de desarrolladores.
- Diferente lugar geográfico
- Voluntarios
- Diferente Idioma (Ingles, Español, Etc)
- No hay un diseño escrito sino un problema por resolver.

Bajo este modelo se ha producido software de gran calidad, como:

- El núcleo Linux.
- Apache
- Samba
- The Gimp

1.1.7. Ventajas

Internacionalización: dado el carácter global de los proyectos, siempre hay quien haga las traducciones.

Reutilización de código e ideas: siendo el código libre, cualquiera puede coger partes de otros proyectos o ver como han resultado otros los distintos problemas.

Reutilización de componentes: dado el carácter cooperativo, los proyectos tratan de producir componentes reutilizables como puede ser aspell, un corrector ortográfico de gran calidad. El equipo de desarrollo de aspell hace el motor de corrección ortográfica, y todos los demás proyectos pueden beneficiarse de ello. Y los usuarios solo tienen que preocuparse de instalar diccionarios una vez para todos los programas.

Rapidez de desarrollo: son decenas, cientos y a veces miles las personas que colaboran en determinadas fases del desarrollo.

Robustez: las extensivas pruebas de funcionamiento entre los usuarios realimentan a los desarrolladores en ciclos increíblemente cortos.

Extensibilidad: cualquiera puede desarrollar nuevas funcionalidades. La calidad de su desarrollo y su aceptación por parte de los usuarios valida la incorporación del nuevo código a la distribución oficial.

Soporte técnico:

GNU/Linux cuenta con el mayor soporte técnico del mundo. La comunidad de usuarios, que va desde meros aficionados y estudiantes a curtidísimos profesionales y consultores del mundo UNIX, tiene una predisposición a la colaboración, especialmente a través de los diferentes medios que ofrece Internet, que permite obtener tiempos de respuesta a cuestiones de tipo servicio técnico muy superiores a los servicios convencionales.

Soporte técnico a través de canales comerciales en crecimiento explosivo: autónomos, pymes y grandes empresas del entorno GNU/Linux y últimamente compañías como HP e IBM disponen de programas de servicio técnico 24h, 365 días al año.

La disposición del código fuente permite a la empresa atacar los hipotéticos problemas con sus propios recursos, bien sea solucionando



'bugs' o bien añadiendo o extendiendo funcionalidades de las aplicaciones. Esto no es posible en entornos comerciales sin una penalización temporal o económica, o aún ambos, normalmente inabordable.

1.1.8. Mitos

Dado que cualquiera puede modificarlo y redistribuirlo, al final hay un montón de versiones distintas y es todo un caos.

Hacer muchas variantes del mismo software, no es rentable, supone mucha duplicación de esfuerzos y confusión. Eso es precisamente el tipo de cosa que se trata de evitar desde el software libre. Nadie en su sano juicio bifurca un proyecto sin una buena razón. Y en cualquier caso siempre se puede coger el software directamente del equipo de desarrollo oficial.

Si no tienes conocimientos suficientes, ni tiempo muchas veces, ¿de qué te sirve el código fuente?

Hay gente que sí tiene el conocimiento y el tiempo, y cuando se descubre algún fallo salta a la luz pública enseguida. De esta forma, todos los usuarios que lo utilizan son conscientes de esos fallos de seguridad y pueden obrar en consecuencia. En materia de seguridad, la falta de transparencia sólo perjudica a los usuarios, porque puede resultar mucho más difícil descubrir las vulnerabilidades del software que estén utilizando.

Al estar el código fuente disponible los intrusos tienen ventajas, ya que pueden descubrir antes los fallos y aprovecharse de ellos.

Cierto, pero hay algo más sobre este tema, que desde los sectores contrarios al software libre no se comenta. No sólo los intrusos los descubren, sino que también hay gente dedicada a la "caza" de bugs que descubre los fallos, los pone en conocimiento de los usuarios y los arregla, con lo cual los usuarios pueden obrar en consecuencia (por otro lado, al ser conscientes del fallo de seguridad y gracias a la disponibilidad del código fuente, es posible que cualquier persona con los conocimientos adecuados pueda arreglar el

fallo y poner la versión ya reparada a disposición de los usuarios). Con software propietario, habría que esperar a que la empresa sacara el parche o Service Pack correspondiente y no hay que olvidar que esto lo hará cuando le salga económicamente rentable. Generalmente se espera a tener corregidos una determinada cantidad de errores y sólo entonces se libera el correspondiente Service Pack, en lugar de solucionar los fallos cuando se detectan. De este tipo de políticas también salimos perjudicados los usuarios.

Por otro lado, en el mundo propietario, cuando alguien descubre un agujero de seguridad, puede explotarlo durante mucho tiempo sin el conocimiento del afectado o el productor del software. En software libre, se descubren mas agujeros que salen pronto a la luz pública y son tapados rápidamente. En el peor de los casos no es mas que un intercambio de ventajas.

1.2 GNU/Linux

1.2.1 Historia de GNU/Linux

A finales de la década de los 60, Ken Thompson y Dennis Ritchie, de los Bell Laboratories (AT&T), desarrollaron un sistema operativo que denominaron jocosamente UNIX. Esto fue una broma, porque el proyecto en el que había estado trabajando antes Thompson se llamaba MULTICS.

Era un sistema elegante y, sobre todo, sencillo. Originalmente corría en una máquina (DEC PDP-7) que era menos potente que cualquier teléfono móvil actual. El sistema primeramente se escribió en lenguaje ensamblador, específico para el PDP-7. Si bien esto le brindaba una gran eficiencia al código, tenía un inconveniente muy serio: el sistema no podía transportarse a otro ordenador distinto; si querían correr UNIX en un ordenador nuevo, tendrían que reescribirlo íntegramente. Claramente, este enfoque no era muy práctico. Tuvieron una idea: crear un lenguaje ensamblador 'abstracto', de manera que, reescribiendo UNIX en dicho lenguaje, el esfuerzo de transportarlo a una nueva arquitectura debería de ser mínimo. Así nació el lenguaje de programación C. Y, con él UNIX



fue reescrito, convirtiéndose así en el primer sistema operativo transportable de la historia.

1.2.3. Richard Matthew Stallman.

En el año 1984, Stallman decidió iniciar el proyecto GNU (GNU's Not UNIX), un proyecto cuya finalidad era proporcionar un sistema operativo similar a UNIX, pero con una licencia que impidiese una 'vuelta a la oscuridad' como la que sufrió el propio UNIX. Dicha licencia se llamó GPL (GNU Public License) y le confiere al software la propiedad de ser libre y permanecer libre.

1.2.4 El proyecto GNU.

Stallman comenzó a construir una herramienta fundamental para el sistema: el compilador para el lenguaje C (gcc, de GNU C Compiler). Esta pieza de software se ha convertido probablemente en el nexo de unión más importante de todo el software libre. Con el tiempo, fue el compilador utilizado por Linus Torvalds para desarrollar el famoso kernel Linux. Un porcentaje muy alto de todo el código asociado al software libre, está escrito en C. Éste lenguaje es, pues, la lengua nativa del sistema, como podría haber sido el esperanto, en el marco de las lenguas humanas. Y, gracias a Stallman, el compilador gcc (y el resto del sistema) es, literalmente, patrimonio de la humanidad.

1.2.5. ¿Por qué llamarlo "GNU/Linux" en lugar de "linux"?

Porque de esa manera se reconoce explícitamente que el sistema operativo no solo es el núcleo linux, sino muchas otras piezas que se escribieron con anterioridad y sin cuya existencia nunca habría sido posible construirlo ni tener algo funcional en nuestros equipos. Todas esas piezas juntas forman un sistema GNU, que es como se denomina al proyecto para construir un sistema operativo totalmente libre iniciado a mediados de los ochenta por la Free Software Foundation. De hecho, el sistema GNU podría tener en un futuro no muy lejano otros núcleos, como el Hurd, con los cuales los usuarios podrán elegir entre sistemas GNU/Linux o GNU/Hurd. Lo realmente importante es disponer de un

sistema operativo libre, no el núcleo, el escritorio o el subsistema gráfico que lleve (y que irá cambiando con el tiempo).

1.2.5 ¿Qué son las "distribuciones" de GNU/Linux?

Una distribución es un modo de facilitar la instalación, la configuración y el mantenimiento de un sistema GNU/Linux. Al principio, las distribuciones se limitaban a recopilar software libre, empaquetarlo en disquetes o CD-ROM y redistribuirlo o venderlo.

Ahora las grandes distribuciones -RedHat, SuSE, Caldera, Mandrake, Corel Linux, TurboLinux...- son potentes empresas que compiten entre sí por incluir el último software, a veces también software propietario, con instalaciones gráficas capaces de autodetectar el hardware y que instalan un sistema entero en unos cuantos minutos sin apenas preguntas.

Entre las distribuciones de GNU/Linux, destaca el proyecto Debian/GNU. Debian nace como una iniciativa no comercial de la FSF, aunque luego se independiza de ésta y va más allá del propio sistema GNU/Linux. Es la única de las grandes distribuciones que no tiene intereses comerciales ni empresariales. Son sus propios usuarios, muy activos, quienes mantienen la distribución de modo comunitario, incluidas todas sus estructuras de decisión y funcionamiento. Su objetivo es recopilar, difundir y promover el uso del software libre. Reúne el mayor catálogo de software libre, todos ellos probados, mantenidos y documentados por algún desarrollador voluntario.

En una distribución hay todo el software necesario para instalar en un ordenador personal; servidor, correo, ofimática, fax, navegación de red, seguridad, etc.

Finalmente tenemos los CD Lives, como Knoppix, Pequelín, etc.

Se espera que los próximos años tengamos nuestra propia distro (Distribución), desarrollada por el Grupo de Usuarios de Software Libre – Somos Libres.



1.3 Filosofía Unix

¿Por qué tuvo tanto éxito el enfoque de UNIX? Aparentemente, su simplicidad fue un factor decisivo. En su diseño, sus creadores antepusieron la facilidad de comprensión a la eficiencia, de manera que era fácil entender el código y, por ende, adaptarlo a las necesidades de otros. UNIX no es una reliquia del pasado; de hecho, la mayor parte de los sistemas operativos actuales son una evolución de UNIX (incluidos MS-DOS y Windows NT/2000/XP). Por eso conviene conocer los principios en los que se fundamente, puesto que esos mismos principios estarán presentes (de una u otra manera) en los sistemas que hoy podamos manejar.

1.3.1. Todo es un archivo.

Esta idea, propia de la orientación a objetos (si bien la precede), consiste en que la unidad básica para la interacción con el sistema es una entidad llamada archivo que, como los archivos en papel, puede abrirse, leerse, avanzar hojas hacia delante y hacia atrás, escribir en él, y cerrarse. Este modelo tan sencillo puede parecer ingenuo, pero ha probado ser extremadamente valioso. Permite a un programa acceder transparentemente a un documento de texto o a un puerto de comunicaciones.

1.3.2. La navaja suiza.

UNIX incorpora un conjunto de herramientas que guardan cierta analogía con una navaja multiusos. Son simples, pero hacen muy bien su trabajo. En lugar de construir programas muy complejos, UNIX proporcionaba muchas pequeñas herramientas, y un esquema para poder combinarlas de forma efectiva. Este diseño escala muy bien, permitiendo al sistema crecer, incorporar nuevas herramientas y, a la vez, ser compatible hacia atrás.

1.3.3. Manual en línea.

Cuando Thompson y Ritchie estaban desarrollando UNIX, solicitaron a sus jefes un ordenador más potente (un DEC PDP-11), a cambio de desarrollar un sistema completo de

tipografía (no les dijeron nada acerca de UNIX). Con el nuevo ordenador desarrollaron UNIX sobre C y, Joe F. Ossanna desarrolló troff (de typesetting run-off). Este sistema fue incluido en el propio UNIX, de manera que el manual del sistema fue escrito con él, estando disponible en línea desde entonces (a través del programa man).

1.4 Distribuciones GNU/Linux en Español

Una distribución Linux, o distribución GNU/Linux (abreviada con frecuencia distro) es un conjunto de aplicaciones reunidas por un grupo, empresa o persona para permitir instalar fácilmente un sistema Linux (también llamado GNU/Linux). Son 'sabores' de Linux que, en general, se destacan por las herramientas para configuración y sistemas de paquetes de software a instalar.

Existen numerosas distribuciones Linux. Cada una de ellas puede incluir cualquier número de software adicional (libre o no), como algunos que facilitan la instalación del sistema y una enorme variedad de aplicaciones, entre ellos, entornos gráficos, suites ofimáticas, servidores web, servidores de correo, servidores FTP, etcétera.

La base de cada distribución incluye el núcleo Linux, con las bibliotecas y herramientas del proyecto GNU y de muchos otros proyectos/grupos de software, como BSD.

Usualmente se utiliza la plataforma XFree86 o la Xorg para sostener interfaces gráficas (esta última es un fork de XFree86, surgido a raíz del cambio de licencia que este proyecto sufrió en la versión 4.4 y que lo hacía incompatible con la GPL).

¿Qué es TumiX GNU/Linux?

Tumix GNU/Linux es una distribución de software libre que se desarrolla en el Perú, incluye el kernel Linux 2.6.10 y está basada en la distribución Slackware (la primera distribución Linux), escritorio KDE y una gran cantidad de software académico, ofimática, multimedia y de redes, la versión actual es la 0.9, liberado el 15 de Junio del 2005. TumiX GNU/Linux se distribuye bajo la licencia GNU GPL.



TumiX CD Live

TumiX GNU/Linux, arranca y funciona a partir de un CD, por consiguiente puede utilizar TumiX GNU/Linux desde donde quiera, en casa, la universidad, el colegio, la oficina. Esta interesante forma de funcionar permite al usuario la utilización de esta distribución en el computador de una forma transparente y no tiene que configurar e instalar GNU/Linux, tan solo tiene que usarlo.

¿Qué significa TumiX?

TumiX es una combinación de palabras y significados, entre Tumi (Cuchillo de sacrificio ritual, utilizado en la Cultura Chimú Perú) y la terminación "X" por el sistema X windows (X es el encargado de visualizar la información gráfica y es totalmente independiente del sistema operativo) que utilizan Linux y UNIX, TumiX nace en la Tacna Perú (ciudad al sur del Perú), pero ve la Luz por primera vez en Piura Perú (ciudad al norte de Perú, donde se desarrollo la Cultura Chimu)

Propósito

Tumix GNU/Linux, nace para fomentar el uso y desarrollo del software libre en el Perú, desde una perspectiva constructivista para promover la investigación, innovación y desarrollo alrededor del software libre. Es una iniciativa de la comunidad de Software Libre del Peru (softwarelibre.org.pe), organización que promueve y difunde el software libre en el Perú y Latinoamérica.

Importante

TumiX se usa libremente se distribuye gratuitamente y tiene licencia GNU GPL.

Descarga

TumiX esta disponible como una imagen ISO. Descarga el ISO, comprueba el md5sum del ISO y posteriormente queme el ISO en un CD, luego bootea desde el CD TumiX GNU/Linux. Asegurate que tu BIOS de tu Computador, este configurada como prioridad de arranque el lector de CD.

Download TumiX GNU/Linux Gracias a la colaboración de la comunidad de Software Libre en el Perú y Empresas Peruanas de Hosting, Tumix GNU/Linux lo pueden descargar de los siguientes URL:

<http://tumix.aniak.org/> (Oficial)

<http://www.virtualorbis.com/tumix/> (Oficial)

<http://tumix.cipher.com.pe/ISO/> (Oficial)

Versión Bittorrent de tumiX Gnu/Linux (Oficial)

<http://linuxtracker.org/download.php?id=390&name=tumiX.torrent>

Universitat Politècnica de Catalunya

<http://ftp.calui.info/pub/distribucions/distribucions-live-cd/tumix/>

MD5Sum

md5sum 8a868bac9ac9fd1fb4e84c3053d1391f

Más Información

<http://tumix.softwarelibre.org.pe>

Notas finales:

Por favor cualquier consulta, duda, sugerencia a este Capitulo serán siempre bienvenidas si fuese así por favor enviar un email a: **daniel@somoslibres.org**
Prof. Daniel Alejandro Yucra Sotomayor
Profesor del Curso GNU/Linux nivel Administrador.
Colaborador: Ing. Julio Sotomayor Abarca

Grupo de Usuarios de Software Libre Perú – GUSL (www.somoslibres.org)

Por un futuro libre y abierto.